Experiment 'p_6_2_h_is_sorted_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 1
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 1

Problem Specification

Problem name: p_6_2_h_is_sorted_array
Natural language statement: null
Method signature: p_6_2_h_is_sorted_array(inputs: array<int>) returns (is_sorted: bool)

Ensures

- `is_sorted == (forall i :: 0 <= i < inputs.Length - 1 ==> inputs[i] <= inputs[i + 1])`

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_6_2_h_is_sorted_array(inputs: array<int>) returns (is_sorted: bool)

The method should respect the following contract:

ensures is_sorted == (forall i :: 0 <= i < inputs.Length - 1 ==> inputs[i] <= inputs[i +
    ↪ 1])

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnO4t0Vwge1bCJTI3vgBJJ52ZtgaA
Model: gpt-5.5-2026-04-23
Created at: 1780662999
Finish reason: stop
Usage: 296 tokens in, and 3530 tokens out

```
{
  var i := 0;
  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length ==> inputs[j] <= inputs[j +
        ↪ 1]
  {
    if inputs[i] > inputs[i + 1] {
      is_sorted := false;
      assert 0 <= i < inputs.Length - 1;
      assert !(forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] <= inputs[j + 1]) by {
        if forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] <= inputs[j + 1] {
          assert inputs[i] <= inputs[i + 1];
          assert false;
        }
      }
      return;
    }

    var old_i := i;
```

```
    assert inputs[old_i] <= inputs[old_i + 1];
    i := i + 1;
    assert forall j :: 0 <= j < i && j + 1 < inputs.Length ==> inputs[j] <= inputs[j + 1]
        ↪ by {
      forall j | 0 <= j < i && j + 1 < inputs.Length
        ensures inputs[j] <= inputs[j + 1]
      {
        if j == old_i {
          assert inputs[j] <= inputs[j + 1];
        } else {
          assert j < old_i;
          assert inputs[j] <= inputs[j + 1];
        }
      }
    }
  }

  is_sorted := true;
  assert forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] <= inputs[j + 1] by {
    forall j | 0 <= j < inputs.Length - 1
      ensures inputs[j] <= inputs[j + 1]
    {
      assert !(i < inputs.Length - 1);
      assert inputs.Length - 1 <= i;
      assert j < i;
      assert j + 1 < inputs.Length;
      assert inputs[j] <= inputs[j + 1];
    }
  }
}
```

Final Program

```
method p_6_2_h_is_sorted_array(inputs: array<int>) returns (is_sorted: bool)
       ensures is_sorted == (forall i :: 0 <= i < inputs.Length - 1 ==> inputs[i] <=
           ↪ inputs[i + 1])
{
  var i := 0;
  while i < inputs.Length - 1
    invariant 0 <= i <= inputs.Length
    invariant forall j :: 0 <= j < i && j + 1 < inputs.Length ==> inputs[j] <= inputs[j +
        ↪ 1]
  {
    if inputs[i] > inputs[i + 1] {
      is_sorted := false;
      assert 0 <= i < inputs.Length - 1;
      assert !(forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] <= inputs[j + 1]) by {
        if forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] <= inputs[j + 1] {
          assert inputs[i] <= inputs[i + 1];
          assert false;
```

```
      }
    }
    return;
  }

  var old_i := i;
  assert inputs[old_i] <= inputs[old_i + 1];
  i := i + 1;
  assert forall j :: 0 <= j < i && j + 1 < inputs.Length ==> inputs[j] <= inputs[j + 1]
      ↪ by {
    forall j | 0 <= j < i && j + 1 < inputs.Length
      ensures inputs[j] <= inputs[j + 1]
    {
      if j == old_i {
        assert inputs[j] <= inputs[j + 1];
      } else {
        assert j < old_i;
        assert inputs[j] <= inputs[j + 1];
      }
    }
  }
 }

 is_sorted := true;
 assert forall j :: 0 <= j < inputs.Length - 1 ==> inputs[j] <= inputs[j + 1] by {
  forall j | 0 <= j < inputs.Length - 1
    ensures inputs[j] <= inputs[j + 1]
  {
    assert !(i < inputs.Length - 1);
    assert inputs.Length - 1 <= i;
    assert j < i;
    assert j + 1 < inputs.Length;
    assert inputs[j] <= inputs[j + 1];
  }
 }
}
```

Total Token Usage

Input tokens: 296
Output tokens: 3530
Reasoning tokens: 2997
Sum of 'total tokens': 3826

Experiment Timings

Overall Experiment started at 1780662999129, ended at 1780663093099, lasting 93970ms (93.97 seconds)
Iteration #1 started at 1780662999129, ended at 1780663093099, lasting 93970ms (93.97 seconds)